



GATEWAY TECHNICAL

Natural Language Processing



UNIVERSITY OF FLORIDA



0. The Oldest Data Problem

Language: The Oldest Data Problem





1. Introduction to NLP

Why NLP? Humans infer meaning through context, tone, and shared knowledge.

- Computers see only **characters and punctuation**, no inherent meaning.
- The same words can mean different things in different contexts:
 - “I went to the **bank**.” (finance)
 - “I was fishing near the **bank**.” (river)
- NLP bridges this gap by giving language **structure** machines can learn from.



Figure: Humans understand meaning
→ machines see raw symbols

"I made her duck"



"I made her duck"



"I made her duck"



Too far?

Natural Language Processing (NLP) = turning language into structured data



Goal

Represent meaning in a form that algorithms can reason about

Typical Steps

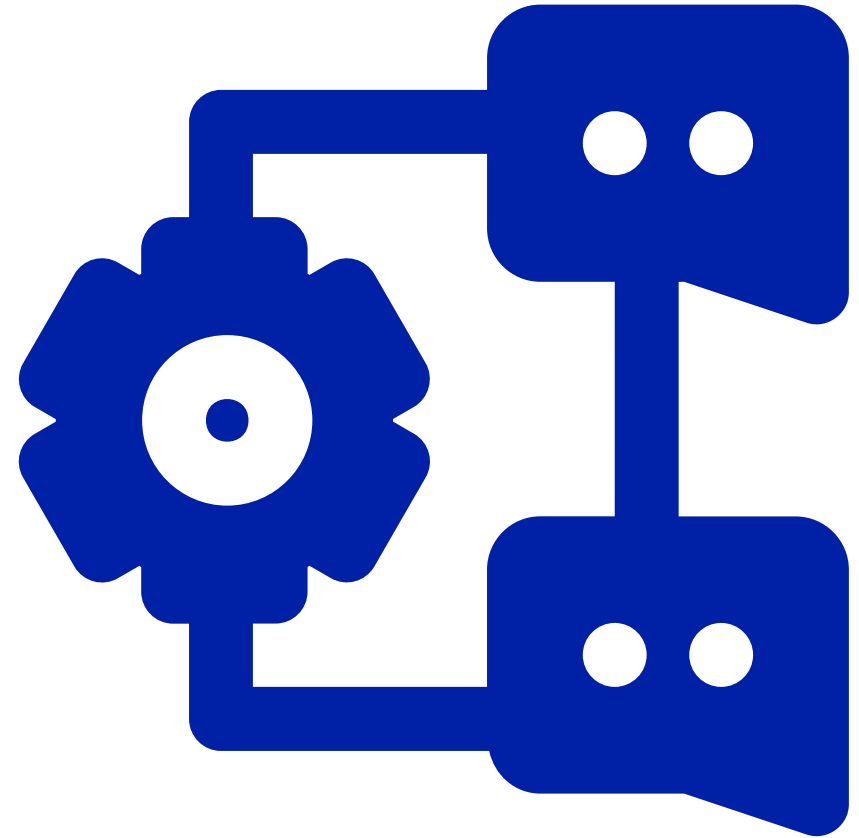
1. **Preprocessing:** Clean and segment text
2. **Representation:** Convert words into vectors or features
3. **Modeling:** Train systems to classify, summarize, translate, or predict

Decision

Every step is a **design decision** about what information to keep or discard.

Representation Design: Raw Text to Structured Tokens

- Reduces **spurious variation** (encoding, casing, punctuation)
- Standardizes **linguistic units** for modeling
- Balances **signal preservation** vs. **noise removal** (task-dependent)



Text Hygiene



Encoding / Unicode normalization

Normalize characters (é vs. e + ´)

Case handling

Lowercasing simplifies vocab;
case may matter for entities

Punctuation, emojis, URLs, HTML

Decide what to strip vs. retain

Contractions

Expand “can’t” → “cannot” for clarity

Sentence Segmentation & Document Structure

- 
- Break text into **sentences** for finer analysis
 - **Tools:** Sentence boundary detection (e.g., NLTK, spaCy)
 - Paragraph and section structure can be used as higher-level features.

Figure: Example of sentence segmentation



2a. Tokenization and Lexical Normalization

Tokenization Approaches

Tokenization: Approaches

- **Rule-based:** Whitespace/punctuation; fast but brittle
- **NLTK / spaCy:** Handles abbreviations and multi-word tokens
- **Subword tokenization:** Mitigates OOV issues (BPE, WordPiece, SentencePiece)
- **Byte-level tokenization:** Robust across languages and emojis
- Transformers *require* subword tokenization

Tokenization: Why It Exists

Goal

Map text to learnable units while preserving meaning signals

Granularities

Word / subword / byte

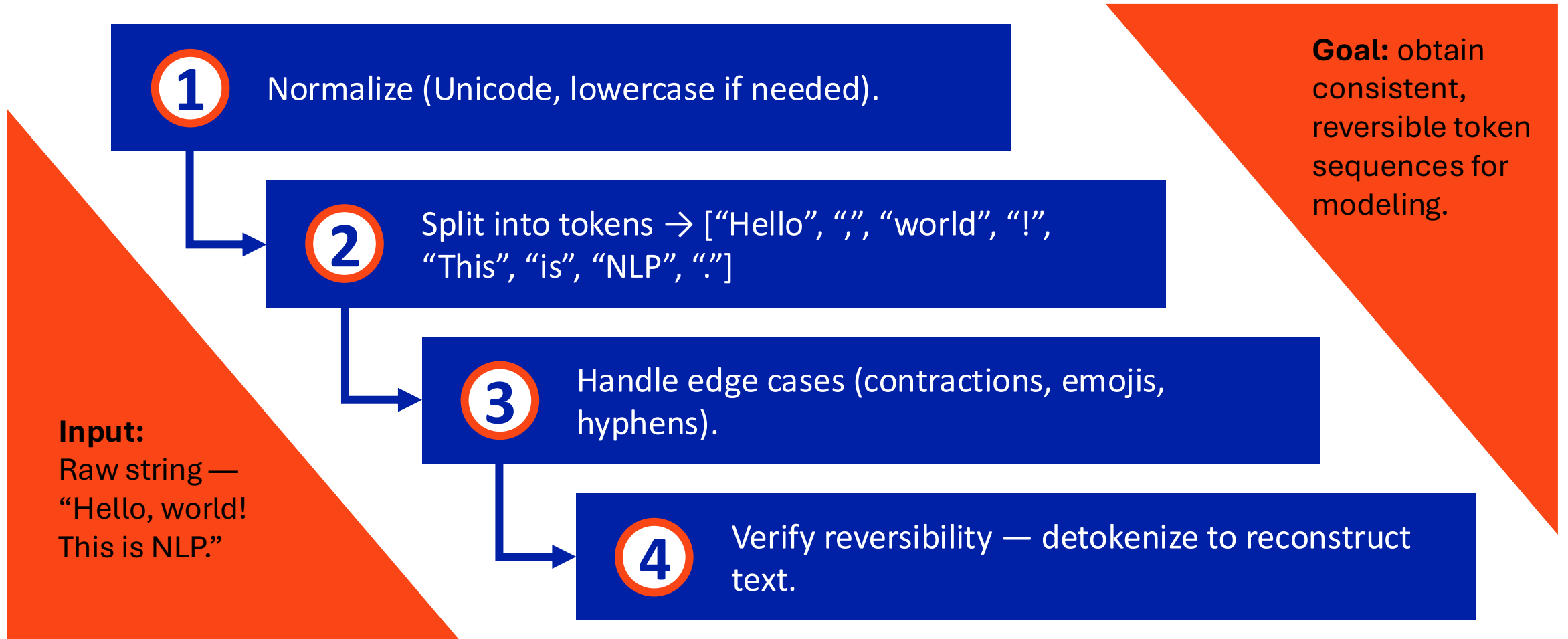
Languages matter

Morphology and whitespace conventions shape choice

Normalization and detokenization

Unicode, punctuation, emojis, and reversibility

The supervised learning workflow follows a standard process



Conceptual Mechanics: BPE, WordPiece, Unigram

BPE

Frequent pair merges →
subunits (data-driven morphology)

WordPiece

Merges chosen to **maximize corpus likelihood**
(probabilistic)

Unigram (SentencePiece)

Start with large vocab, prune to best subunits

All aim to balance **coverage, compression,**
and learnability.

Tokenization & Tasks: Measuring the Impact (Conceptual Demo)



What changes when
you switch regimes?

OOV rate, average tokens/doc, “meaning granularity”

Effects on models

- **Sparse models:** Vocab size and sparsity patterns shift
- **Dense/embeddings:** Neighborhood structure and analogy quality change

Tokenization is Task-Dependent



Spam Detection

Lowercase and simplify
aggressively



Named Entity Recognition (NER) or Contracts

Preserve case and
punctuation



Transformers

Use subword tokenization



2b. Normalization

Why We Normalize Language

- **Goal:** Reduce **lexical variance**, map multiple surface forms to a single meaning
- Helps models generalize across inflection, casing, and common words
- Each normalization step trades **semantic nuance** for **statistical efficiency**.



Stopword Filtering

Function words

Removes **function words** that carry grammar but little content

Examples

the, and, of, to, but

Caution

Keep semantically critical words such as *not*, *never*, *no*

Custom stopwords lists improve domain performance (e.g., legal, medical).

Figure: Stopword removal impact on vocabulary size

Stemming

Heuristic truncation

Removes suffixes to reach root forms

Examples

- *running* → *run*
- *connected* → *connect*
- *happily* → *happi*

Fast but crude, may yield non-words
(*comput*)

Figure: Example stemming transformations

Lemmatization

Maps words to their **dictionary base form** (lemma)

Uses **part-of-speech tagging** + morphological analysis

Examples

- “running” → *run* (*verb*)
- “better” → *good* (*adjective*)

Requires lexical resources (e.g., WordNet, spaCy, Stanza)

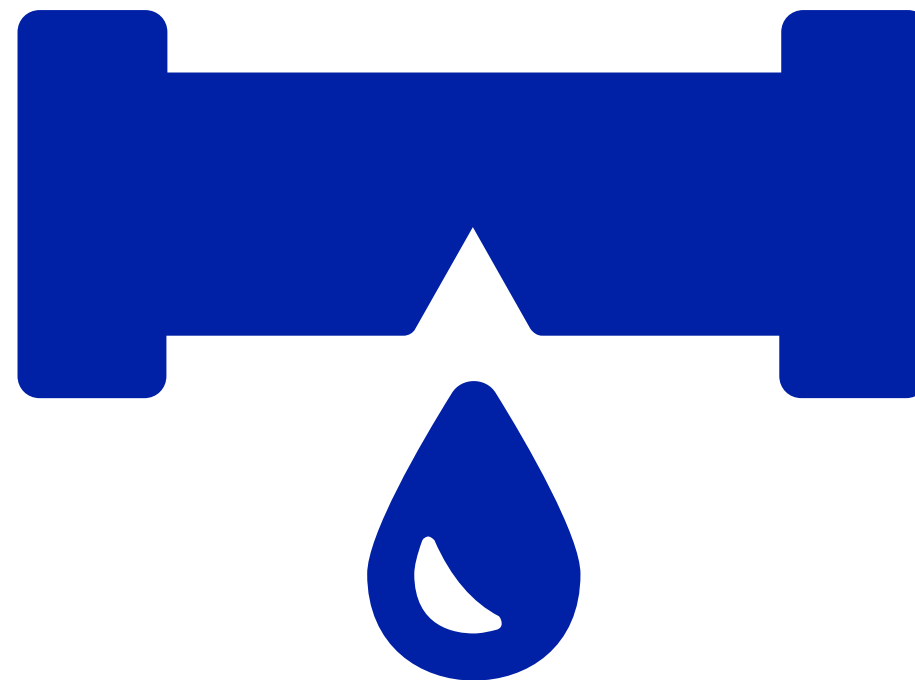
Figure: Lemmatization example with POS awareness

Choosing the Right Level of Normalization

Technique	Linguistic Awareness	Output Type	Speed	Best Use Case
Stopword Removal	None	Reduced vocab	Very fast	Simplify BoW / TF-IDF
Stemming	Morphological (surface)	Truncated roots	Fast	Search, indexing
Lemmatization	Morphological + syntactic	True lemmas	Moderate	Analytics / interpretability

Data Leakage Warning

- Fit preprocessing (vectorizers, normalizers) on **training data only**.
- Apply learned transformations to validation/test sets.
- **Example:** Computing IDF on full corpus leaks test information.





3. Linguistic Feature Extraction

Part-of-Speech (POS) Tagging & Dependency Basics

POS tagging

Assigns grammatical roles
(noun, verb, adjective, etc.)

Uses

- **Feature engineering**
(e.g., ratio of nouns to verbs)
- **Filtering by word type**
(e.g., adjectives for sentiment)

Dependency parsing (intro only)

Shows grammatical relationships

Example: “The dog chased the ball” →
subject = dog, object = ball

Figure: POS tags and dependency relations
in a sample sentence

Named Entity Recognition (NER)

Identifies **entities** such as people, organizations, locations, dates, and amounts

Example

“Apple acquired Beats for \$3B in 2014” →

Apple = ORG, Beats = ORG, \$3B = MONEY,
2014 = DATE

Applications

- **PII redaction**
(masking names, emails, addresses)
- **Business analysis** (e.g., counting ORG mentions in financial filings)

Figure: Highlighted entities in text

N-Grams, Frequency, and Co-Occurrence

N-grams

Word sequences capturing short context

- **Unigrams:** “I”, “love”, “pizza.”
- **Bigrams:** “love pizza.”
- **Trigrams:** “I love pizza.”

Frequency counts

Identify dominant words or phrases

Co-occurrence

Detect words appearing together often

Example: “fraud” with “card,” “transaction”

Figure: Frequency distribution
and co-occurrence network

Lexical Diversity and Readability



Type–token ratio (TTR)

- Unique words ÷ total words
- **Caution:** Shorter texts appear more diverse.

Readability metrics

Flesch Reading Ease, Flesch–Kincaid Grade Level

Applications

- Loan narratives → complexity and literacy
- Marketing copy → repetition vs. variety

Features are Task-Dependent

Not all linguistic features are equally valuable.

Sentiment analysis

Adjectives and adverbs

Fraud detection

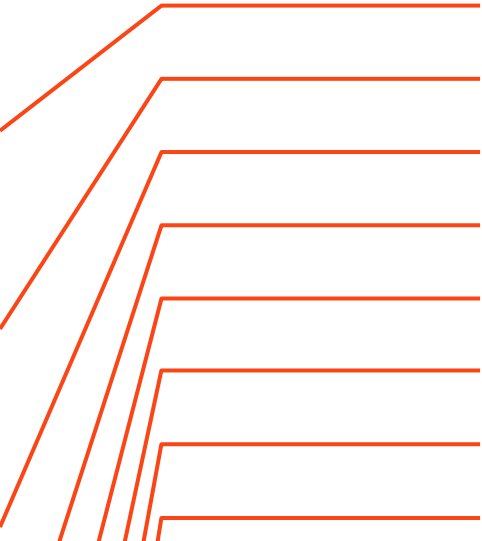
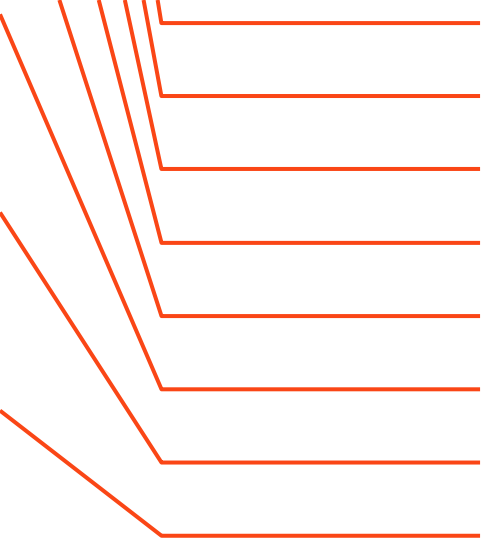
Money, dates (NER)

Readability

Sentence length, diversity



Figure: Examples of task-specific linguistic features



Bridge: From Linguistic Features to Learned Representations

What we've learned so far



4. Representations

Why We Use Embeddings

- 
- Deep learning models **cannot process raw text**.
 - Embeddings convert **discrete symbols** (words, pixels, notes) into **continuous vectors**.
 - These vectors encode **semantic and relational structure** that models can learn from.
 - Every data type (**text, audio, video**) has its own embedding model suited to its modality.

Figure: Illustration of three embedding pipelines for text, audio, and video. Each transforms raw data into numerical vector spaces the network can understand.

Bag-of-Words (BoW) and TF-IDF

Bag-of-Words

- Represents text as simple word counts
- Intuitive, interpretable, but ignores order and context

TF-IDF (Term Frequency–Inverse Document Frequency)

- Adjusts frequency by informativeness
- $(= ())$ where N = total docs, n_t = docs containing term t
- $\text{TF-IDF} = \text{TF} \times \text{IDF}$

Considerations

Vocabulary growth, n-gram ranges, and hashing trick

Limitations

Sparse, memory-heavy, no semantic relationships or polysemy handling

Low-Rank Semantics: LSA / SVD

Latent Semantic Analysis (LSA)

Applies Singular Value Decomposition (SVD) to TF-IDF

Latent Structure

Compresses sparse vectors into lower dimensions to reveal **latent structure**

Example

“car” and “automobile” appear close in reduced semantic space.

Limitation

Context-free; smooths but doesn't disambiguate meaning

Static Embeddings (Word2Vec and GloVe)

Word2Vec

Learns dense vectors by predicting context (CBOW) or word (Skip-Gram)

Case handling

Learns embeddings via matrix factorization of word co-occurrence statistics

Punctuation, emojis, URLs, HTML

Captures analogical and semantic relationships (e.g., *king – man + woman $\approx$ queen*)

Contractions

One vector per word $\rightarrow$ no contextual differentiation (*bank* = river or finance)

Figure: Word embeddings plotted in 2D semantic space

Contextual Embeddings (Preview)

ELMo

Generates context-dependent word vectors (same word → different vectors)

BERT

Bidirectional transformer model capturing context on both sides

Sentence-BERT

Produces sentence-level embeddings for similarity and retrieval

Key advancement

Meaning changes dynamically with context; embeddings adapt to task

Figure: Contextual embedding visualization
(same word, different meanings)

What is an Embedding? (Discrete → Continuous)

Purpose

Convert discrete tokens into **continuous vectors** that a model can learn from

Static vs. contextual

One vector per type (**static**) vs.
one per occurrence (**contextual**)

Interpretability vs. power

- **Static** = more interpretable
- **Contextual** = more expressive

Where used

Features for classic ML, similarity search, clustering, and as inputs to deep models

Figure: Contextual embedding visualization (same word, different meanings)

When to Move from Sparse → Dense



Sparse (BoW, TF-IDF)

Small/medium datasets, simple tasks, easy to explain

Dense (Word2Vec, BERT)

Large corpora, semantic or contextual tasks

Rule of thumb

- Overlap-based tasks → sparse
- Meaning-driven tasks → dense



5. Sentiment and Emotion Analysis

VADER (Valence Aware Dictionary and sEntiment Reasoner)

- Designed for **short, social text** such as tweets and product reviews
- Combines **lexicon** and **rules** for handling capitalization, punctuation, and negation



Outputs

Positive, negative, neutral, and **compound** sentiment scores



Strengths

Fast, interpretable, effective for informal text



Limitations

Brittle on long or domain-specific text

TextBlob

Provides **polarity** (-1 negative → +1 positive) and **subjectivity** (0 objective → 1 subjective)



Strengths

Quick implementation, readable API,
suitable for beginners



Limitations

Not tuned for specialized domains or
informal tone

NRCLEX (Emotion Analysis)

- Classifies words into **emotion categories**: anger, fear, joy, trust, etc
- Extends sentiment beyond positive/negative to multidimensional emotion



Strengths

Highly interpretable and explainable for non-technical audiences



Limitations

Context-blind; words like “killer app” may trigger incorrect emotions

Critical Caveats in Lexicon-Based Analysis

Domain shift

Lexicons don't generalize well
(e.g., medical vs. financial texts).

Sarcasm and irony

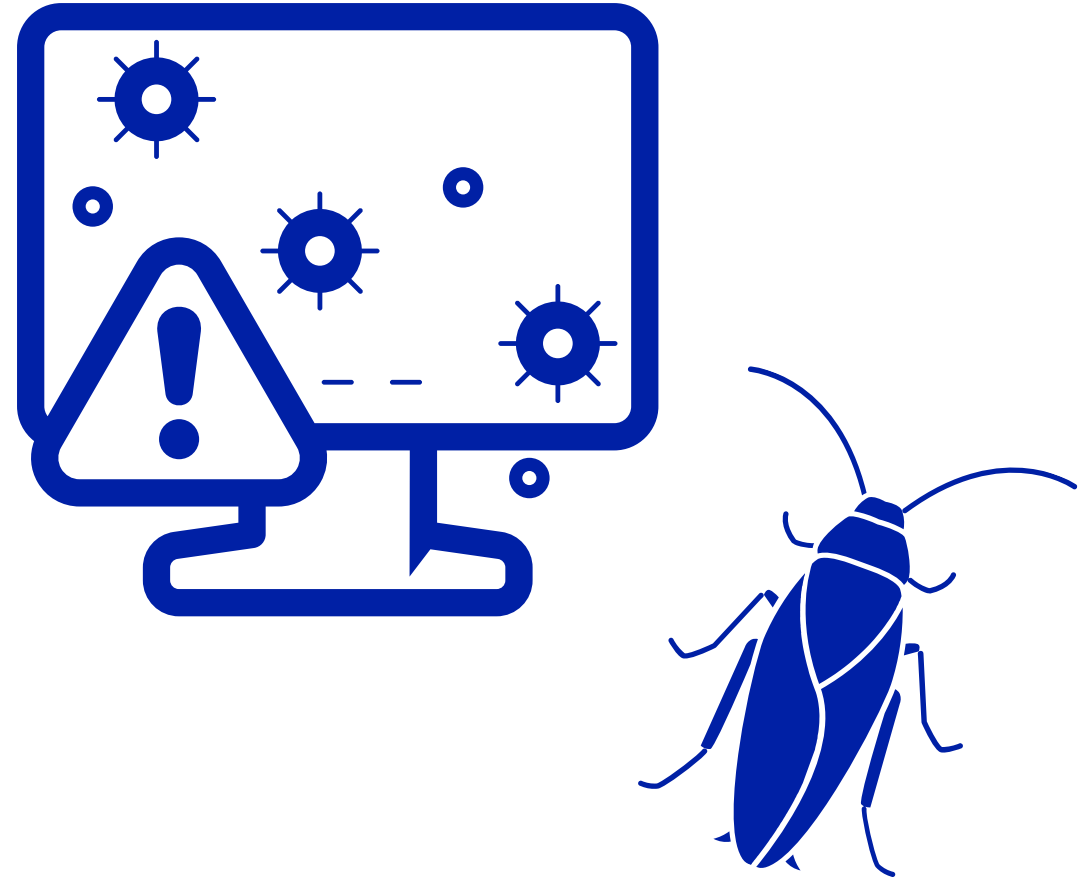
Models misread statements like:
"Great, another bug."

Multilingual limitations

Most sentiment tools focus on English.

Bias

Lexicons may encode cultural or linguistic
stereotypes.



Sentiment $\neq$ Ground Truth

- Sentiment scores are **signals**, not objective truth.
- Best used as **features** for modeling or exploratory data analysis
- Always validate sentiment against domain-specific benchmarks or labeled examples.

Your directions were great

Your directions were awful

Your directions were not great

Your directions were impressively awful

I enjoyed being lost after receiving your directions

Figure: Comparing sentiment scores with true customer feedback labels



6. Quick Modeling Loop (TF-IDF → Logistic Regression)

Building a Minimal Text Classification Pipeline

- 
- 1 Step 1: Clean text, tokenize, remove stopwords, lemmatize.
 - 2 Step 2: Vectorize, convert text into TF-IDF feature vectors.
 - 3 Step 3: Train, fit a **logistic regression** classifier on TF-IDF features.

Why logistic regression?

- Simple and interpretable baseline
- Produces class probabilities for threshold tuning
- Reveals most predictive words via coefficient weights

Evaluation for Text Classification: Conceptual Overview

Stratified splits

Preserve class ratios in train/val/test

F1 family

- **Macro-F1:** Unweighted mean across classes (fair to minority)
- **Micro-F1:** Global average (tracks overall accuracy)
- **Weighted-F1:** Class-size-weighted mean (compromise)

Calibration

Predicted probabilities $\neq$ true likelihoods
Use reliability plots; consider Platt or isotonic calibration

Confusion matrix

Summarize TP / FP / FN / TN to reason about errors

Precision vs. Recall

Reliability of positives vs. coverage of positives

PR vs. ROC

Prefer **PR** for imbalanced data;
ROC can be overly optimistic

Report set

Confusion matrix + Macro-F1 + PR-AUC
(and ROC-AUC for completeness)

Evaluation for Text Classification: Quick Reference

Scenario / Question	Recommended Metric(s)	Why
Classes imbalanced	PR-AUC, Macro-F1	Focuses performance on minority class
Overall performance, balanced data	Accuracy, Micro-F1	Reflects global correctness
Need to tune thresholds	Precision/Recall, F1, PR curve	Make trade-offs explicit
Risk of false negatives is costly	Recall, Macro-F1	Maximize positive coverage
Limited review capacity (costly false positives)	Precision, Precision@k	Prioritize reliability
Use of probabilities downstream	Calibration curves, Brier score	Ensure probability reliability
Model comparison across runs	Fixed split + seed , same metrics	Apples-to-apples evaluation

F1 Variants (When to Use)



Macro-F1

Fairness across classes

Micro-F1

Large, balanced datasets

Weighted-F1

Mild imbalance, single score

Error Analysis and Robustness (Fast Diagnostics)



Inspect errors

Top **false positives/negatives** with short text snippets

Slice performance

By length, presence of negation, emoji usage, entity density

Stress tests

- Simple perturbations to test brittleness
- Remove emojis / randomize casing / inject 5% typos → note Δ in F1

Domain shift check

Train on subset A; test on subset B (e.g., electronics → home goods)



7. Reproducibility, Ethics, and Bridge to LLMs

Classic Pipeline vs. LLMs

Baseline pipeline

TF-IDF + Logistic Regression =
transparent, efficient, reproducible

LLMs

Use embeddings and contextual understanding for greater
semantic depth

Key contrasts

- **Interpretability:** Logistic regression > LLM
- **Accuracy on nuanced tasks:** LLM > logistic regression
- **Cost & compute:** Logistic regression $\ll$ LLM

All aim to balance coverage, compression, and
learnability.

Figure: Comparison of classic NLP
pipeline vs. LLM-based workflows

Pipeline Discipline and Artifacts (Reproducible Baseline)

Pipelines

Fit on **train only**, transform **val/test**
(discipline via
Pipeline/ColumnTransformer concept)

Version and hash

Record library versions and dataset checksums

Persist artifacts

Vectorizer, vocab/IDF, model, splits, seeds

Comparability

Reuse the same artifacts when benchmarking
LLM approaches later

- Set **random seeds** for tokenizers, splits, and classifiers.
- Track **dataset and pipeline versions** for consistent results.
- Maintain **prompt logs** (in later modules) for transparency and auditing.
- Reproducibility builds scientific and operational credibility.

Bridge to LLMs: What Changes vs. What Stays



Changes

Subword/byte tokenization, **contextual embeddings**, attention over long sequences

Stays the same

Hygiene (Unicode, casing), leakage discipline, **evaluation and calibration**, error analysis, artifact versioning

In LLMs

Compare TF-IDF/logistic baseline vs. **prompting, fine-tuning**, and **RAG** on the **same task**



Hands-On Exercise

Hands-On Exercise Overview



dataset provided

- `loan_text.csv`: Loan narratives with approval/denial outcomes

Goal

Build an end-to-end text classification pipeline

Hands-On Exercise Steps



1

Step 1: Full Cleaning Pipeline

- Normalize text (Unicode NFC/NFKC).
- Handle URLs, emojis, and contractions.
- Tokenize + lemmatize using spaCy or NLTK.
- Apply stopwords filtering when appropriate.

Hands-On Exercise Steps



1

Step 1: Full Cleaning Pipeline

2

Step 2: POS and NER Features

- Extract **part-of-speech (POS)** distribution (e.g., % adjectives, % verbs).
- Use **Named Entity Recognition (NER)** to identify MONEY, ORG, DATE, etc.
- **Optional:** Engineer numeric features such as entity counts, text length, and lexical diversity.

Hands-On Exercise Steps



1

Step 1: Full Cleaning Pipeline

2

Step 2: POS and NER Features

3

Step 3: Sentiment and Emotion Features

- Apply **VADER** → compound polarity.
- Apply **TextBlob** → polarity and subjectivity.
- Apply **NRClex** → emotion scores (anger, joy, trust, etc.).
- Store all outputs as numeric features for classification.

Hands-On Exercise Steps



1

Step 1: Full Cleaning Pipeline

2

Step 2: POS and NER Features

3

Step 3: Sentiment and Emotion Features

4

Step 4: Vectorization

- Create **BoW** and **TF-IDF** matrices from cleaned text.
- Optionally apply **LSA (SVD)** for dimensionality reduction.
- Discuss vocabulary size, sparsity, and interpretability trade-offs.

Hands-On Exercise Steps



1 Step 1: Full Cleaning Pipeline

2 Step 2: POS and NER Features

3 Step 3: Sentiment and Emotion Features

4 Step 4: Vectorization

5 Step 5: Train Logistic Regression

- Train **logistic regression** on TF-IDF features.
- **Evaluate using:**
 - Accuracy
 - Precision, recall, and F1 (macro-F1 emphasized)
 - Confusion matrix visualization

Hands-On Exercise Steps



1 Step 1: Full Cleaning Pipeline

2 Step 2: POS and NER Features

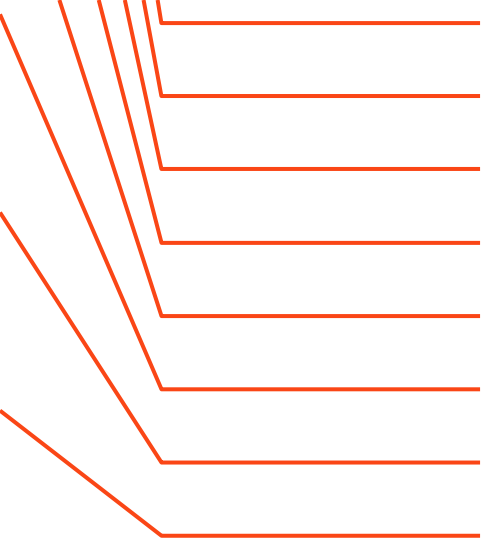
3 Step 3: Sentiment and Emotion Features

4 Step 4: Vectorization

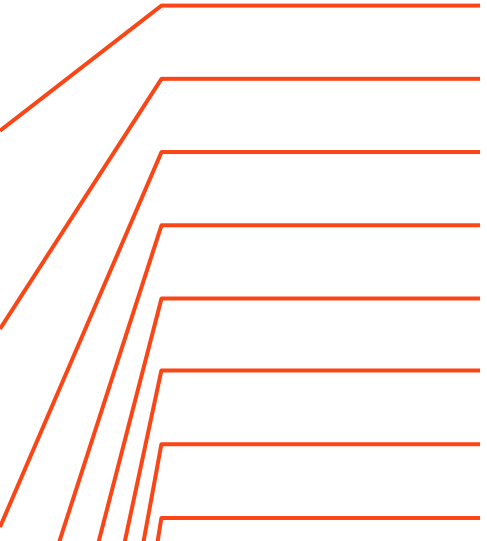
5 Step 5: Train Logistic Regression

6 Step 6: Compare Patterns and Visualize

- Compare sentiment/emotion distributions across predicted classes.
- Visualize frequent words, entity mentions, and emotion trends.
- (Generally speaking) Prefer bar plots over word clouds for clarity and comparability.
- Emphasize correct labeling, scales, and context for interpretability.



Discussion



- 1 When are sparse models (TF-IDF + logistic regression) good enough?
- 2 When might you need embeddings or deep learning models?
- 3 Discuss trade-offs among interpretability, complexity, and compute resources.



Appendix: Additional Slides

Word2Vec: Example of a Word Embedding Algorithm

Learns word meaning from **context** using simple neural objectives

- **CBOW:** Predict word from context
- **Skip-Gram:** Predict context from word

Produces vectors where **semantic similarity = geometric proximity**

Classic analogies emerge

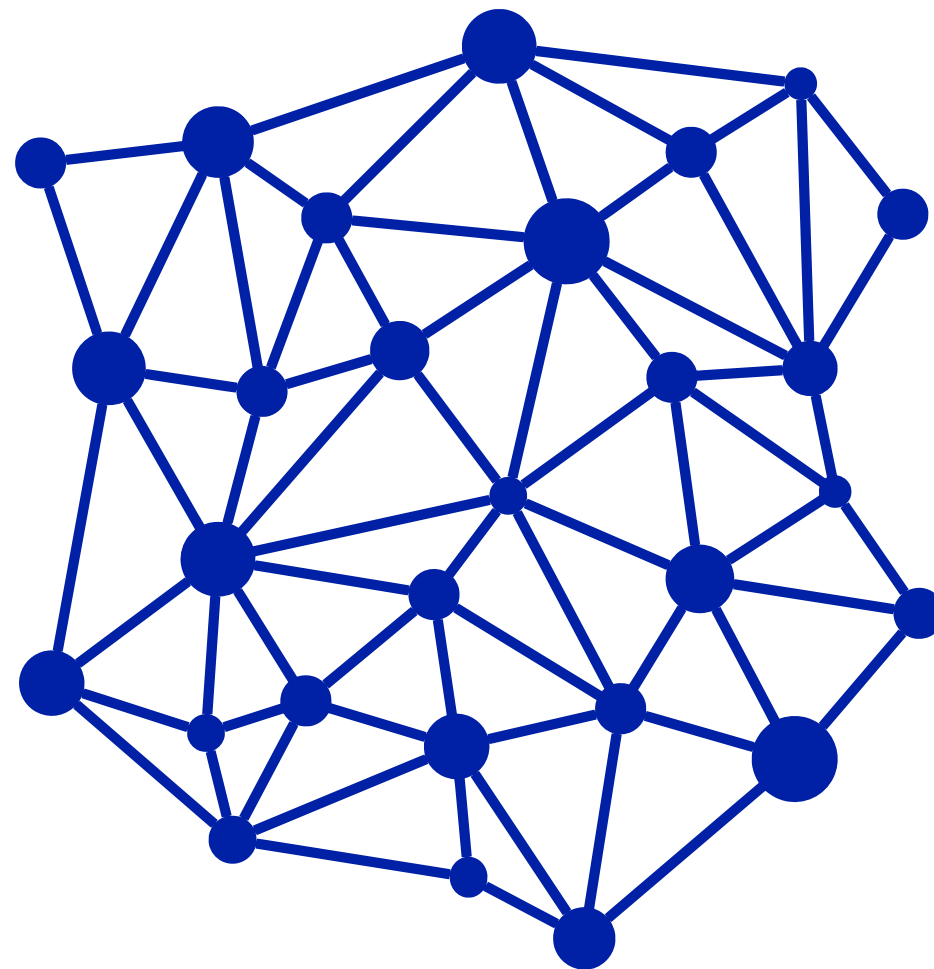
king – man + woman $\approx$ queen

Visualization

Words cluster by meaning in 2D space

Side Note: LLMs Learn Their Own Embeddings

- LLMs include **trainable embedding layers** optimized during pretraining.
- These embeddings adapt to the **task and data**, not fixed like Word2Vec.
- Embedding dimensionality grows with model size:
 - **GPT-2 Small (117 M params):** 768 dims
 - **GPT-3 (175 B params):** 12 288 dims
- Trade-off → higher dimension = richer representation but greater cost





Appendix: Stylometry

Stylometry and Author Identification (Mini-Segment)



Goal

Identify likely author by writing style (not content)

Features

Function-word ratios, punctuation patterns, sentence length variance, **character n-grams**, POS trigrams

Protocol cautions

- Split data by author (avoid cross-document leakage)
- Control topical cues; prefer style markers over content words

Handling Class Imbalance

Weighted loss

`class_weight="balanced"` adjusts penalties by class frequency

Resampling strategies

- Oversample minority class (e.g., SMOTE)
- Undersample majority class to rebalance

Trade-offs

Improves fairness and recall but may reduce accuracy or stability